1st class

Windows – MSDOS Kernel

- Computation

Sequence of instruction —--> Program
Set of information -> Data

- Components of Computer hardware

Central Unit
ALU
Registers
Cache Memory

Primary Memory -> RAM
Secondary Memory -> Storage device

- Modern Hardware : Parallel Computing

Socket

- Memory Hierarchy

Registers
Cache
Main Memory

Permanent Storage

Find the specification of the pc
How would you select a new laptop or pc

Rooted tree file system
2nd Class

command	Full form	work
pwd	present working directory	used for current location
ls	List every thing	Give the list of
clear		Clear the sceen

Single dot denote current directory
Double dot denote files in the one level above

Tab completion

ls -l give me detail me in a column format

Each file have three category of the permission
Owner–group– user

ls -l | piping – it is used to concate with the next command
Example ls -l | less

Less is a pager it is used to save the result in buffer and show that page by page

3rd Class

Ls -l | grep Do

Give the list of the files of with name start with do

In the page buffer if we press / it open the search bar

printenv this is print the information of the system

echo $ {Program} this give the path of the program

Vim .bashrc open the bashrc -> (press I to open it in write mode) -> edit -> esc -> exit/ insert mode -> :w to write the change on the system -> :q-> back to $

ls -l ./Do*

Alias myls=“”

4th Lecture

lscpu :- give all information about the cpu.

lscpu > about-my-computer.txt :- make a file named about-my-computer.txt and content will be all everything that lscpu will give

-h make things human readable

>> appending thing at the last

uname -a description about the kernel

lshw complete information about the system

lsblk :- information about the block devices

top :- show the current programs running

    kill {Pid from the top}

ps -sf :- shapsort of the programs

journalctl -b :- log of system boot files

systemctl list-units --type=service --state=running :- list of the all the currently running service

file manipulation
    create
    rename

move
copy
delete
edit

touch {name} :- used to create empty files

echo work with input sting and cat works on input file

cp -v {file to be copied} {new name of the file} :- copy the file with new name

moving and renaming is the same thing,

mv -v {file name} {new location}

rm -v :- it is used to delete the file.

Lecture 5th

mkdir {name}:- this create a directory of name =”name”

Bash Scripting

- -seq
- -for loop
- -run through files in a directory
- -execute bash scripts

owner - group - users

users are ppl who can login into the cluster but not attached to the group

.sh :- this is bash file.

```
for FILE in ./*
do
	echo $FILE
done
```

this is a bash program this give the list of all files in that directory

:! command this is use to use temporary command prompt in the vim

shebang

#! /usr/bin/env bash vs #! /usr/bin/bash

chmod:- this command is used to change the permissions of the files

SSH :- Secure SHell
We have a server, that has some well established servers, local and public, for us it is local and can not use outside iiser bunch of clients, they can use ssh protocol to login to this server.

server sit and acts as mediator for computer

SSH

- asymmetric cryptography
- -ssh server
- ssh client
- password authentication v/s key-based authentication

pubilc and private key

how password is shared with private and public key.

how to login with encrypted credentials.

LDAP

Lecture 6th

Version Control and Collaboration

Problem with Version control

Files named:

- Final.c
- final_v2.c
- final_really_final.c

will take the help of other system like github or something that can do this management

No clear history
Difficult collaboration
Easy to loose

Local Version Control System

RCS-Idea Keep patch sets that can be used to recreate a file

forward and backward direction is used

patch is the something that have the information about what was changes

Centralized version Control(SVN)

svn is type of version control system

shared repository

developer developer developer

if two ppl working on same file and one ppl checkin then person 2 have to first check out the new version created by person 1 then and only person can do new thing

if shared repository loose all the data

net connection is required

Branching is expensive. This becomes a tree

Decentralized version control

Distributed version control
every clone has full history
fast, local commits

Gits stores snapshots, not diffs

nearly all operations are local

Three Stages of the Files

📄 Git Basics – Exam Ready Notes (DOCX Content)

1. Git

Git is a **distributed version control system** used to track changes in files and manage source code.

Command to check Git installation:

```
git
```

---

2. Working Directory

Command:

```
pwd
```

Purpose: Displays the present working directory.

---

3. Git Configuration (Global)

Set username:

```
git config --global user.name "Ajay Kasaudhan"
```

Set email:

```
git config --global user.email
ajay.kasaudhan@students.iiserpune.ac.in
```

Purpose: Identifies the author of commits.

Configuration file:

`~/.gitconfig`

---

4. Create Project Directory

```
mkdir dspractice
cd dspractice
```

Purpose: Creates a directory for Git practice.

---

5. Initialize Git Repository

```
git init
```

Purpose: Initializes an empty Git repository.
Result: Creates a hidden `.git` directory.
Default branch: `master`

---

6. Hidden Files

```
ls -a
```

Purpose: Displays hidden files like `.git`.

---

7. `.git` vs `.gitconfig`

`.git`	`.gitconfig`

Repository-specific	Global configuration
Inside project folder	Inside home directory
Stores commits & branches	Stores username & email

---

8. Create and Edit File

```
touch rajat
vim rajat
```

Purpose: Creates and edits a new file.

---

9. Git Status

```
git status
```

Purpose: Shows:

- Untracked files
- Staged files
- Commit status

---

10. Staging Files

```
git add rajat
```

Purpose: Adds file to the staging area.

---

11. Commit

```
git commit -m "no"
```

Purpose: Saves staged changes permanently to the repository.
`-m` specifies commit message (mandatory).

---

12. Git Workflow (Very Important)

```
Working Directory → Staging Area → Repository
```

---

13. Common Errors

- ❌ `git -status` (invalid command)
- ❌ `git commit -m` (missing message)

⭐ One-Line Exam Summary

Git tracks file changes by moving files from **working directory** → **staging area** → **committed history**.

Lecture 7th

How to do undo things

restore it is use to undo the things

git restore - - staged {file} #for unstaging
git restore {file} # to discard changes

Access previous versions

git checkout <commit-hash>
git checkout <tag>

git restore –source=HEAD-1 file.txt

git show <commit-hash> : </path/to/file>
git show HEAD~1:</path/to/file>

git diff <commit-hash> <commit-hash> - - /path/to/file

git have something called switch

NOTE - git refre to the pwd

Branching

This is use when we need to set the project in two different direction
Branch is a pointer to a commit

```
v5
↑       v32
v4        ↑
↑       v31
v3
↑
v2
↑
v1
```

HEAD points to the current branch
Creating branches are cheap

What is the name of you recurrent branch

merge conflict

- same line changed in two branches
- git cannot guess intents

Resolution Step
-Edit file manually

1. What is Git?

Git is a **Distributed Version Control System (DVCS)** used to track changes in files, manage source code, and enable collaboration.

Key features:

- Distributed (every user has full repository)
- Fast and reliable
- Tracks complete history
- Supports branching and merging

---

2. Basic Git Architecture

Git has **three main areas**:

1. **Working Directory**
   → Actual files you edit
2. **Staging Area (Index)**
   → Files selected for the next commit
3. **Repository (HEAD)**
   → Saved snapshots (commits)

```
Working Directory → Staging Area → Repository
```

---

3. Important Git Terms (Very Important)

Term	Meaning
Repository	Project tracked by Git
Commit	Snapshot of changes
Branch	Pointer to a commit
HEAD	Current commit reference
Detached HEAD	HEAD not pointing to a branch
Merge	Combine branches
Checkout / Switch	Move between branches/commits

---

4. Basic Git Commands

Initialize Repository

```
git init
```

Check Status

```
git status
```

Add Files to Staging

```
git add file.txt
git add .
```

Commit Changes

```
git commit -m "message"
```

View Commit History

```
git log
git log --oneline
```

---

5. `git add` and `git commit`

`git add`

- Moves files from **working directory** → **staging area**
- Requires **exact filename**

```
❌ git add h
✅ git add h.txt
```

`git commit`

- Saves staged files as a snapshot
- Commit message should be **clear and meaningful**

Example:

```
git commit -m "Added input validation"
```

---

6. `git commit -a`

```
git commit -a -m "message"
```

✔ Automatically stages **modified tracked files**
❌ Does **not** add new untracked files

---

7. `git restore`

Unstage a file

```
git restore --staged file.txt
```

Discard local changes

```
git restore file.txt
```

---

8. Branching in Git

Create Branch

```
git branch feature
```

Switch Branch

```
git checkout feature
# OR
git switch feature
```

Create & Switch Together

```
git switch -c feature
```

---

9. Why Branches Are Used

- Parallel development
- Feature isolation
- Safe experimentation
- Team collaboration

---

10. Detached HEAD State (Very Important)

How it happens

```
git checkout <commit-hash>
```

Meaning

- HEAD points directly to a commit
- No branch is active
- Commits can be lost if not saved to a branch

Exit Detached HEAD

```
git switch -
# OR
git checkout master
```

---

11. Merging Branches

Merge Feature into Master

```
git checkout master
git merge feature
```

Types of Merge

- **Fast-forward merge**
- **Three-way merge**

---

12. Merge Conflict

Occurs when:

- Same file
- Same lines
- Different changes

Resolution steps:

1. Open conflicted file
2. Fix manually

3. `git add file`

4. `git commit`

---

13. `git log --graph`

```
git log --graph --oneline --all
```

Shows:

- Branch structure
- Merge history
- Commit relationships

---

14. Common Mistakes (Exam Highlight)

Mistake	Reason
`git add h`	Git needs exact filename
File not added	File never created
Detached HEAD confusion	Checked out commit, not branch
`commit -a` missing file	File was untracked
Poor commit message	Bad practice

---

15. File Naming (Linux + Git)

- `"hello this is file"` is **one filename**
- Spaces require quotes
- Prefer:

```
hello_this_is_file.txt
```

---

16. Best Practices (Exam Point)

- Small commits
- Clear commit messages
- Use branches for features
- Avoid `--amend` after push
- Check `git status` frequently

---

17. One-Line Exam Answers

- **Git**: Distributed version control system
- **Commit**: Snapshot of tracked files

- **Branch**: Movable pointer to commit
- **HEAD**: Current commit reference
- **Detached HEAD**: HEAD not on branch
- **Merge**: Combining branches

---

18. Typical Exam Question Patterns

Q1. Difference between `git add` and `git commit`

✔ `git add` stages changes
✔ `git commit` saves snapshot

Q2. What is Detached HEAD?

✔ HEAD pointing directly to a commit instead of a branch

Q3. Why use branches?

✔ Parallel development and isolation

Q4. What does `git commit -a` do?

✔ Commits tracked modified files only

Lecture X

Git push and pull

Lecture 8th

Program Execution

Interpretation

Compilation

Virtual Machine

Elements of a Imperative Programming Languages

- Variables
-

Python is a dynamic typed language

Types are associated with values (objects), not the variable name. Type checking happen at run time , not before execution .

Every value in Python is an object.

Each values have following features.

- scope
- type
- life time
-

Set operation

a & b intersection

a | b union

a - b subtraction

Binary Data Type

Used for raw binary data and I/O

- bytes - immutable sequence of bytes
- bytearray - mutable sequence of bytes
- memoryview - zero-copy data

Special Data Type : None

User-Defined Data Types

Create using classes

Default argument

```
def power( x , exponent = 2):
        return x ** exponent

power(3)      -> 9
power(3,3)      -> 27
```

Important Note on Default Arguments

```
def add_item(item , items=[ ]):
        items.appends



def add_item_new(item, items = None):
...     if items is None:
...             items = []
...     items.append(item)
...     return items
```

```
...
>>> add_item_new(3)
[3]
>>> add_item_new(6)
[6]
>>>
```

Yield Keyword

yield is used inside a function to a value at a time.

A function containing yield is called a generator

Comprehensions: the big idea